Documentação da Suíte de Testes — CuidaMed API

Testes de carga, estresse, integração e regressão com k6 · 07/11/2026

Este documento apresenta os testes executados com o **k6 v2.3.0** na API do CuidaMed, aplicativo de lembretes de medicamentos para idosos e cuidadores. A API é feita em **Java 25 com Spring Boot 4.1.1** e usa **PostgreSQL 18**. Os testes acessam a API como o aplicativo faz, criam as próprias contas de teste e as apagam no fim: nenhum dado de paciente real é lido ou alterado.

Ambiente: Notebook Intel Core i3-1115G4, 3,7 GB de RAM, Windows 11. A API e o k6 rodaram na mesma máquina, com um banco PostgreSQL separado só para os testes (cuidamed_k6).

Estrutura dos scripts

- **carga.js** — teste de carga: o uso normal do paciente (abrir o Início com dados, remédios e histórico; registrar e desfazer a dose).
- **estresse.js** — teste de estresse: o mesmo uso, em degraus acima do normal, para achar o limite e ver se a API se recupera.
- **integracao.js** — teste de integração (E2E): cadastro e login do cuidador e do paciente, vínculo entre os dois, remédio cadastrado pelo cuidador, dose registrada e desfeita pelo paciente, com o estoque conferido a cada passo.
- **regressao.js** — teste de regressão: 34 regras que já funcionavam (acesso, login, cadastro, doses, estoque, histórico por período e compactação) continuam funcionando.
- **comum.js** e **uso-do-paciente.js** — partes compartilhadas: endereço da API, contas de teste e o fluxo do paciente.

Resultados consolidados

CENÁRIO	VUS MÁX.	REQUISIÇÕES	VAZÃO	TAXA DE ERRO	LATÊNCIA MÉDIA	LATÊNCIA P95	CHECKS	STATUS
Carga Uso normal: até 50 pacientes ao mesmo tempo por 3 minutos	50	14.310	50,0/s	0,00%	6,98 ms	10,07 ms	14.160/14.160	**APROVADO**
Estresse Degraus de 25, 50, 100 e 200 usuários, topo mantido e recuperação	200	45.400	104,4/s	0,00%	4,95 ms	7,96 ms	44.800/44.800	**APROVADO**
Integração (E2E) Cadastro, login, vínculo, remédio, dose e estoque	1	19	14,3/s	0,00%	67,25 ms	313,57 ms	22/22	**APROVADO**
Regressão 34 regras de segurança e de negócio	1	60	33,2/s	0,00%	28,16 ms	135,55 ms	34/34	**APROVADO**

VUs: usuários virtuais simultâneos. p95: 95% das respostas foram mais rápidas que esse valor. Na regressão, as recusas que a regra exige (401, 403, 409 e 400) são marcadas como esperadas no script e não contam como erro; qualquer outro status contaria.

Metas (thresholds)

TESTE	MÉTRICA	META	RESULTADO
Carga	Erros HTTP	`rate<0.01`	**Atingida**
Carga	Verificações	`rate>0.99`	**Atingida**
Carga	Tempo do histórico (31 dias)	`p(95)<500`	**Atingida**
Carga	Tempo de resposta	`p(95)<500`	**Atingida**
Carga	Tempo de registrar dose	`p(95)<500`	**Atingida**
Estresse	Tempo de resposta	`p(95)<1500`	**Atingida**
Estresse	Erros HTTP	`rate<0.05`	**Atingida**
Integração (E2E)	Verificações	`rate==1.0`	**Atingida**
Regressão	Verificações	`rate==1.0`	**Atingida**
Regressão	Erros HTTP	`rate==0`	**Atingida**

No estresse, as metas servem de referência para mostrar em que ponto a API deixaria de atender bem.

Destaques

- **Carga sem erros:** com 50 pacientes simultâneos, a API atendeu 14.310 requisições (50,0 por segundo) com 0,00% de erros, latência média de 6,98 ms e p95 de 10,07 ms — muito abaixo da meta de 500 ms. Houve um máximo isolado de 1.402 ms, que não afetou o p95.
- **Estresse sem queda:** no topo de 200 usuários simultâneos a API manteve 0,00% de erros e p95 de 7,96 ms, com 45.400 requisições (104,4 por segundo) e recuperação normal na descida. O limite da API não foi alcançado com o volume que o notebook consegue gerar.
- **Integração completa:** 22 de 22 verificações aprovadas. O p95 maior (313,57 ms) vem do cadastro e do login, que verificam a senha com BCrypt, um cálculo propositalmente lento para proteger as senhas.
- **Regressão:** 34 de 34 regras aprovadas, entre elas: um paciente não vê os dados de outro, a mesma dose não pode ser registrada duas vezes, registrar a dose desconta o estoque e desfazer devolve, e o histórico grande vem compactado (gzip).
- **Dados isolados:** as contas de teste usam e-mails próprios (`k6.…@teste.cuidamed`) e são apagadas no fim de cada teste; o script `limpar-dados-de-teste.sql` remove qualquer resto.
- **Otimizações já aplicadas:** respostas da API compactadas, histórico limitado aos últimos 31 dias no aplicativo, menos consultas ao banco e índices por paciente.

Limitações

- A API e o k6 rodaram no mesmo notebook, sem a latência da internet: em produção os tempos tendem a ser maiores.
- O estresse foi limitado a 200 usuários simultâneos pela memória do notebook (3,7 GB); o script aceita picos maiores (`-e PICO=400`).
- O chat com a IA ficou fora da carga e do estresse, porque cada pergunta é uma chamada paga a um serviço externo.

Como executar

```
cd testes-k6
$env:K6_WEB_DASHBOARD = "true"
$env:K6_WEB_DASHBOARD_EXPORT = "relatorios/carga.html";      k6 run --summary-export relatorios/carga-resumo.json carga.js
$env:K6_WEB_DASHBOARD_EXPORT = "relatorios/estresse.html";   k6 run --summary-export relatorios/estresse-resumo.json -e PICO=200 estresse.js
$env:K6_WEB_DASHBOARD_EXPORT = "relatorios/integracao.html"; k6 run --summary-export relatorios/integracao-resumo.json integracao.js
$env:K6_WEB_DASHBOARD_EXPORT = "relatorios/regressao.html";  k6 run --summary-export relatorios/regressao-resumo.json regressao.js
py gerar-relatorio.py
```

Por padrão os testes usam a API de testes em `http://localhost:8081`. Para outro servidor: `k6 run -e API=https://endereco-da-api carga.js`. Os relatórios HTML de cada teste, gerados pelo k6, estão na pasta `relatorios`.